

VI. Introduction à CSS — Maîtriser le style des pages Web

CSS EN UNE PHRASE

Si HTML décrit *ce que* la page contient, CSS décrit *comment* elle s'affiche : couleurs, tailles, positions, animations, responsive — tout le visuel passe par là.

Sommaire

- **Introduction**
 - 1. Objectifs du chapitre
 - 2. En pratique — situation professionnelle
 - 3. Quand l'utiliser, quand l'éviter
- **A. Qu'est-ce que CSS et pourquoi ?**
 - 1. Rôle de CSS
 - 2. Séparation contenu / présentation
- **B. Les trois méthodes d'intégration**
 - 1. CSS externe (recommandé)
 - 2. CSS interne
 - 3. CSS inline
- **C. Éléments block vs inline**
 - 1. Éléments block
 - 2. Éléments inline
 - 3. Changer le comportement avec display
- **D. Le box-model**
 - 1. Les 4 couches
 - 2. Illustration
 - 3. Raccourcis et formes courtes
 - 4. Les marges négatives et le collapse
- **E. La propriété box-sizing**
 - 1. box-sizing: content-box (défaut historique)
 - 2. box-sizing: border-box (préféré en pratique)
 - 3. Le reset classique
- **Conclusion**
 - 1. À retenir
 - 2. Pour aller plus loin

Introduction

1. Objectifs du chapitre

À l'issue de ce chapitre, vous serez capable de :

- **Expliquer** le rôle de CSS dans la chaîne HTML + CSS + JavaScript et la séparation contenu / présentation.
- **Distinguer** les trois méthodes d'intégration (externe, interne, inline) et choisir la bonne selon le contexte.
- **Distinguer** les éléments **block** et **inline** et prédire leur comportement par défaut.
- **Appliquer** le box-model (content, padding, border, margin) pour calculer la taille finale d'un élément.
- **Reconnaître** la propriété `box-sizing` et son impact sur les calculs de dimension.

2. En pratique — situation professionnelle

Un client vous envoie un design Figma. Il faut le traduire en code. Sans maîtrise du box-model et de l'intégration CSS, vous passez 2h à chasser un bug de 4 pixels qui vient en fait de la différence `content-box` vs `border-box`. Cette leçon vous donne les bases qui évitent des heures de debug.

3. Quand l'utiliser, quand l'éviter

✓ Utiliser pour	✗ Éviter quand
CSS externe (<code><link></code>) en production (cache, réutilisation)	<code>style=""</code> inline en prod (sauf retouches JS dynamiques)
<code>box-sizing: border-box</code> universel dès le début d'un projet	Calculer manuellement <code>width + padding + border</code>
Comprendre block / inline avant Flexbox ou Grid	Apprendre Flexbox sans maîtriser le box-model

A. Qu'est-ce que CSS et pourquoi ?

1. Rôle de CSS

CSS (*Cascading Style Sheets* — feuilles de style en cascade) est un langage de **présentation**. Il prend les éléments HTML bruts et leur applique :

- des **couleurs** et typographies ;
- des **dimensions**, marges et espacements ;
- des **mises en page** (flexbox, grid, positionnement) ;
- des **animations** et transitions ;
- des **variantes** selon la taille d'écran (responsive).

CSS a été créé en **1994** par **Håkon Wium Lie** au CERN. Il est aujourd'hui maintenu par le W3C.

2. Séparation contenu / présentation

Le principe fondateur : le **HTML structure** le contenu, le **CSS présente** ce contenu. Bénéfices :

- **Maintenabilité** — une seule feuille CSS stylise 100 pages HTML.
- **Réutilisabilité** — les mêmes classes s'appliquent à plusieurs projets.
- **Performance** — le navigateur met en cache le fichier CSS, évitant des re-téléchargements.
- **Collaboration** — le designer peut travailler la feuille CSS sans toucher au contenu HTML, et inversement.

♦ La métaphore de la maison

- **HTML** = la structure (murs, toit, pièces).
- **CSS** = la décoration (peinture, mobilier, disposition).
- **JavaScript** = l'électricité et la plomberie (interactivité).

Les trois ensemble forment un site vivant et utilisable.

B. Les trois méthodes d'intégration

1. CSS externe (recommandé)

Le CSS est dans un fichier séparé `.css`, lié depuis le HTML par `<link>`.

```
←!— dans <head> →  
<link rel="stylesheet" href="./assets/styles.css">
```

```
/* styles.css */  
body {  
  font-family: Arial, sans-serif;  
  background-color: #f0f0f0;  
}  
  
h1 {  
  color: #333;  
  text-align: center;  
}
```

✅ **Avantages** : centralisation, réutilisabilité, cache navigateur, séparation nette contenu/style.

2. CSS interne

Le CSS est dans une balise `<style>` placée dans le `<head>`.

```
<head>  
  <style>  
    p { color: blue; font-size: 16px; }  
  </style>  
</head>
```

⚠ **À utiliser seulement** pour :

- des page uniques (mail HTML, landing page one-off) ;
- du CSS *critical* en inline pour accélérer le premier rendu (optimisation avancée).

3. CSS inline

Le CSS est dans l'attribut `style` d'une balise.

```
<p style="color: red; font-size: 20px;">Texte rouge</p>
```

✗ **À éviter** en production, sauf cas vraiment ponctuels. Raisons :

- Répétition massive si plusieurs éléments ont le même style.
- **Spécificité maximale** qui écrase toutes les règles externes (difficile à déboguer).
- Pas de cache, pas de réutilisation.

Règle du pouce

90 % externe, 5 % interne (cas de build SSR/SSG), **< 5 % inline** (ajustements programmatiques par JavaScript).

C. Éléments block vs inline

Avant toute mise en page, il faut comprendre que chaque balise HTML a un **comportement d'affichage par défaut**.

1. Éléments block

- Occupent **toute la largeur** disponible du conteneur parent.
- S'empilent **les uns sous les autres** (retour à la ligne automatique).
- Peuvent recevoir des dimensions (`width` , `height`), margin et padding dans toutes les directions.

Exemples : `<div>` , `<h1>` – `<h6>` , `<p>` , `<section>` , `<article>` , `<header>` , `<footer>` , `<ul>` , `<li>` , `<form>` .

2. Éléments inline

- Occupent **seulement l'espace nécessaire** à leur contenu.
- Se placent **côte à côte** sur la même ligne (flux horizontal).
- **N'acceptent pas** `width` / `height` — seulement margin/padding horizontaux.

Exemples : `<a>` , `<span>` , `<em>` , `<strong>` , `<img>` (cas particulier : inline mais avec `width` / `height`), `<input>` , `<label>` .

3. Changer le comportement avec `display`

La propriété CSS `display` permet de **forcer** un comportement différent :

```
/* Transformer un <span> en block */
.mon-span { display: block; }

/* Transformer un <div> en inline-block (taille sur mesure mais reste en ligne) */
.ma-boite { display: inline-block; }

/* Cacher complètement un élément */
.masque { display: none; }
```

D'autres valeurs importantes : `flex` , `grid` (voir chapitre VI), `inline-flex` , `table` , `contents` .

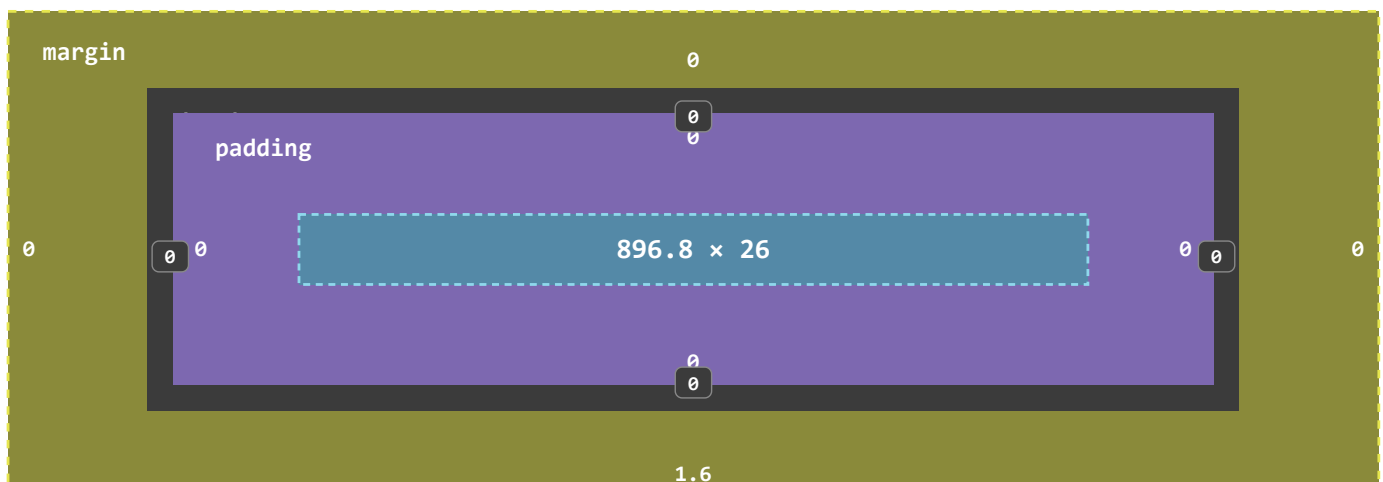
D. Le box-model

Chaque élément HTML (block ou inline) est **rendu par le navigateur sous forme d'une boîte rectangulaire** composée de 4 couches concentriques.

1. Les 4 couches

Couche	Rôle	Exemple de propriété
Content	Le contenu réel (texte, image)	<code>width</code> , <code>height</code>
Padding	Espace intérieur entre le contenu et la bordure	<code>padding: 20px;</code>
Border	Bordure autour du padding	<code>border: 2px solid #000;</code>
Margin	Espace extérieur entre la bordure et les autres éléments	<code>margin: 10px;</code>

2. Illustration



3. Raccourcis et formes courtes

Pour `margin` , `padding` , `border` :

```
margin: 10px; /* 4 côtés */
margin: 10px 20px; /* haut-bas / gauche-droite */
margin: 10px 20px 15px; /* haut / gauche-droite / bas */
margin: 10px 20px 15px 25px; /* haut / droite / bas / gauche (sens horaire) */
```

4. Les marges négatives et le collapse

Attention à deux comportements particuliers :

- Les **marges peuvent être négatives** (`margin-top: -10px`), ce qui tire l'élément vers le haut / la gauche. Utile pour des superpositions maîtrisées, dangereux si utilisé sans réflexion.
- **Collapse des marges verticales** : deux marges verticales adjacentes **fusionnent** et prennent la valeur **la plus grande**, pas la somme. Ex: un élément avec `margin-bottom: 20px` et le suivant avec `margin-top: 30px` → espace effectif = 30px, pas 50px.

⚠ Pas de collapse horizontal, pas de collapse avec padding/border

Le collapse s'applique uniquement aux **marges verticales** (top/bottom) et seulement quand il n'y a **rien** entre les deux (pas de padding, pas de border, pas de contenu inline).

E. La propriété `box-sizing`

C'est la propriété qui **change la définition de** `width` **et** `height`. Deux valeurs possibles :

1. `box-sizing: content-box` (défaut historique)

`width` et `height` s'appliquent **uniquement au contenu**. Le padding et la border **s'ajoutent** au rendu final.

```
.boite {  
  box-sizing: content-box; /* par défaut */  
  width: 200px;  
  padding: 20px;  
  border: 5px solid;  
}  
/* Taille réellement occupée : 200 + 2*20 + 2*5 = 250px ! */
```

2. `box-sizing: border-box` (préféré en pratique)

`width` et `height` incluent **contenu + padding + border**. C'est le mode intuitif.

```
.boite {  
  box-sizing: border-box;  
  width: 200px;  
  padding: 20px;  
  border: 5px solid;  
}  
/* Taille réellement occupée : 200px pile. Le contenu est automatiquement réduit. */
```

3. Le reset classique

Il est devenu **standard** de mettre `border-box` globalement :

```
*, *::before, *::after {  
  box-sizing: border-box;  
}
```

♦ À mettre dès la première règle de tout nouveau projet

Sans ce reset, Bootstrap, Tailwind et la plupart des frameworks CSS se comportent de façon contre-intuitive. La quasi-totalité des projets modernes commencent par `border-box` universel.

Conclusion

1. À retenir

- CSS **sépare présentation et contenu** — maintenabilité, réutilisabilité, performance.
- **3 méthodes d'intégration** : externe (90 %), interne (rare), inline (à éviter).
- Les balises ont un **comportement par défaut** block ou inline — modifiable avec `display`.
- Chaque élément = une **boîte** avec 4 couches : content, padding, border, margin.
- Les **marges verticales fusionnent** (collapse) dans certaines conditions — pas de somme.
- `box-sizing: border-box` universel est un reset quasi-obligatoire en 2026.

2. Pour aller plus loin

- **MDN — Guide CSS**
- **MDN — Le modèle de boîte CSS**
- **MDN — box-sizing**
- **CSS-Tricks — Box-sizing tutorial**